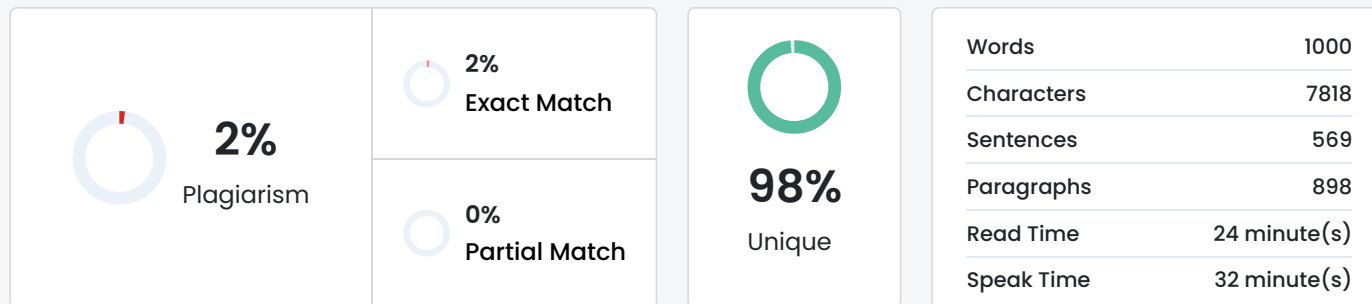


Plagiarism Scan Report



Content Checked For Plagiarism

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ASSIGNMENT REPORT

CSA09 – PROGRAMMING IN JAVA (SLOT B)

SMART HOSPITAL PATIENT APPOINTMENT, PHARMACY INVENTORY AND ALERT NOTIFICATION SYSTEM USING JAVA

Internal Assessment – Common Course Assignment

Student Name

Mann

Register Number

192311090

Team Members

Harsha Kyle; Kumaran S

Faculty Name

Dr. Thalapathy Rajasekaran

Course Code & Name

CSA0911 – Programming in Java

Maximum Marks

100

Department

Department of Computer Science and Engineering

Programme

Academic Year / Batch

2026/2027

Date of Issue

Date of Submission

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

The assigned problem is to design, implement, and evaluate a menu-driven Java application for a Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System. The system must coordinate patient appointments and prescribed medicines while treating doctor slots and medicine stock as limited resources. It must automate patient/doctor/medicine registration, appointment-slot display, booking and cancellation, patient/medicine search and update, waitlist handling, reporting, and appointment/low-stock notifications.

The solution models Patient, Doctor, Appointment, Medicine, Inventory-related data, and Notification behaviour through encapsulated Java classes. Patient categories are handled through inheritance and polymorphism so that category-specific consultation rules can be applied at run time. Java collections are used for record storage and traversal, while built-in and user-defined exceptions protect the application from invalid IDs, duplicate appointments, invalid input, and stock shortages. Multithreading is incorporated for concurrent appointment requests and asynchronous notification dispatch.

1.1 Problem Formulation

The system has two important shared resources: doctor appointment slots and pharmacy inventory. An appointment slot can be assigned to only one active appointment at a time, and medicine quantity must never become negative. Therefore, the application must validate inputs, maintain consistent records, and coordinate concurrent operations. The design uses a synchronized appointment-booking method and a BlockingQueue-based notification worker to address these requirements.

1.2 Inputs

- Patient ID, name, phone, age and category
- Doctor ID, name, specialization, consultation fee and available slots
- Medicine ID, name, type, quantity and reorder level
- Appointment ID or patient/doctor/slot details for appointment operations
- Menu choices and quantities for pharmacy transactions

1.3 Outputs

- Patient and doctor records
- Available doctor slots and appointment status
- Pharmacy inventory and stock alerts
- Appointment confirmation/cancellation notifications
- Waitlist promotion notifications
- Patient-visit and pharmacy-inventory summary report
- Exception messages for invalid or conflicting operations

2. OBJECTIVES AND EXPECTED OUTCOMES

- Develop a reliable menu-driven hospital management application using Java.
- Apply classes, objects, encapsulation, data hiding, constructors, member methods, inheritance and polymorphism.
- Use ArrayList, Set, HashMap, Hashtable, generics, Iterator and ListIterator for collection processing.
- Implement built-in and user-defined exception handling for invalid IDs, duplicate appointments, invalid input and out-of-stock conditions.
- Demonstrate multithreading, thread priorities, synchronization and inter-thread communication.
- Generate meaningful patient, appointment and pharmacy inventory reports.
- Demonstrate an innovative patient-category fee rule and automated low-stock/appointment notifications.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional Requirements

- Register and maintain patient, doctor and medicine records.
- Display available appointment slots and pharmacy inventory.
- Book and cancel appointments and prevent duplicate active bookings.
- Maintain a waitlist for a fully occupied doctor-slot combination and promote the first waiting patient after cancellation.
- Search and update patient and medicine records.
- Track medicine issues/restocking and generate low-stock alerts.
- Generate patient/appointment and inventory utilization summary information.
- Dispatch notifications asynchronously.
- Demonstrate concurrent appointment booking safely.

3.2 Technical Requirements

- Java classes and objects with private data members
- Java Collection Framework with generics and iterators
- Exception handling with custom exception classes
- Thread/Runnable, thread priority, synchronized method and BlockingQueue
- JDBC connectivity for persistent database operations

- Event handling for user-driven application actions
- Git/GitHub for source-code version control

3.3 Constraints

- The reference problem is implemented as an in-memory console application; data is reset after program termination.
- Notification delivery is simulated in the console rather than through a real SMS/e-mail gateway.
- The application assumes valid time-slot strings supplied by the doctor configuration.
- Concurrent booking must preserve a single active booking for a doctor-slot pair.
- Development and submission are subject to the assignment's stated two-day completion window.

3.4 Assumptions

- Each patient has a unique patient ID.
- Each doctor has a unique doctor ID.
- A doctor-slot pair represents one appointment capacity.
- Medicine quantities are non-negative integers.
- The first patient in a waitlist is promoted when a booked slot is cancelled.

4. APPLICATION OF COURSE KNOWLEDGE / CONCEPTS

Course Concept

Project Application

Evidence

CO

Classes & Objects

Patient, Doctor, Appointment, Medicine, Notification and HospitalData

Entity objects created and manipulated

CO1

Encapsulation & Data Hiding

Private fields with public accessors/mutators and behaviour methods

Fields are not directly exposed

CO1

Inheritance

Patient extends abstract Person

Common identity fields reused

CO1

Polymorphism

Patient.getCategory() override and category-specific fee rule

Senior/Child/General behaviour

CO1

ArrayList

Patients, doctors and appointments

Dynamic ordered storage

CO2

Set

Doctor slots

Duplicate time slots prevented

CO2

HashMap

Medicine records keyed by medicine ID

Fast record retrieval

CO2

Hashtable

Waitlist keyed by doctor-slot

Required map collection for waitlist

CO2

Iterator

Medicine inventory traversal

Generic entry iterator

CO2

ListIterator

Patient search/update

Bidirectional collection update mechanism

CO2

Generics

ArrayList<Patient>, HashMap<String, Medicine>, etc.

Type-safe collections

CO2

Custom Exceptions

InvalidPatientException, DuplicateAppointmentException, OutOfStockException, etc.

Meaningful error handling

CO3

Multithreading

BookingTask threads and NotificationWorker

Concurrent operations

CO3

Thread Priority

MAX_PRIORITY and NORM_PRIORITY

Priority demonstration

CO3

Synchronization

synchronized book(), cancel(), Medicine.issue()

Shared-state consistency

CO3

Inter-thread Communication

LinkedBlockingQueue

Notification producer/consumer

CO3

Event Handling

Handles user/menu actions and system events

Event-driven application processing

CO4

JDBC

Database connectivity for hospital records

SQL/database access through JDBC

CO5

5. SOLUTION / DESIGN / METHODOLOGY

The proposed system uses a modular object-oriented design. Entity classes store domain information and expose controlled behaviour. HospitalData acts as the central in-memory service layer. BookingTask represents concurrent booking requests, while NotificationWorker consumes notifications from a shared BlockingQueue. The menu in main() provides the user interface and catches input/operation exceptions.

5.1 System Architecture

5.2 Appointment Workflow

- Validate patient ID → validate doctor ID → validate offered slot → enter synchronized booking section → check active

Matched Source

Similarity 2%
Title:Solved Scenario:You have been tasked with designing a
Jan 19, 2025 — Scenario:You have been tasked with designing a database for a small medical clinic. The clinic needs to keep track of the following information:Patients: ...
[Check](#)
